Evaluate Expression Tree

AlgoExpert · Easy · Binary Trees

The problem

- Leaves are positive ints; internal nodes encode operators -1 '+', -2 '-', -3 '/' (truncating), -4 '*'.
- Evaluate the tree.

Intuition

- Leaves are numbers and internal nodes are operators, so evaluate the children first and combine them.

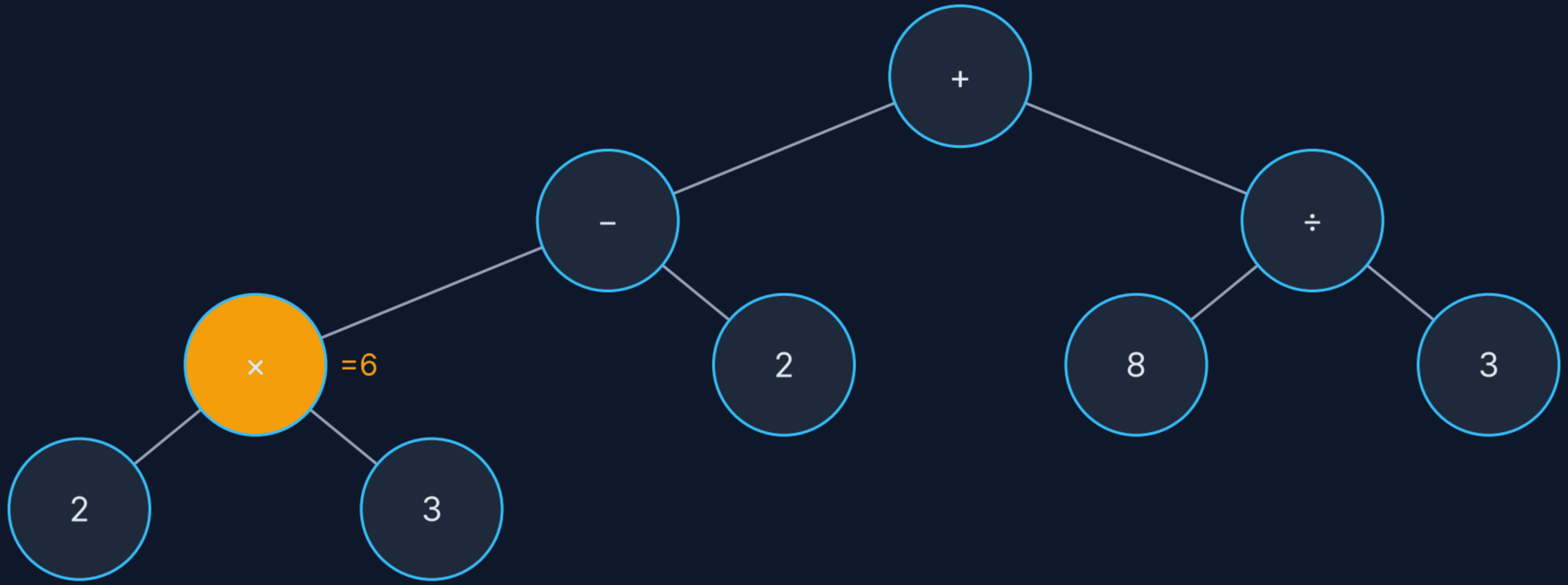
Brute force vs optimal

- There's no faster way: every node must be evaluated once
- Key is post-order: children before the operator
- Recursion depth = height of the tree

Approach

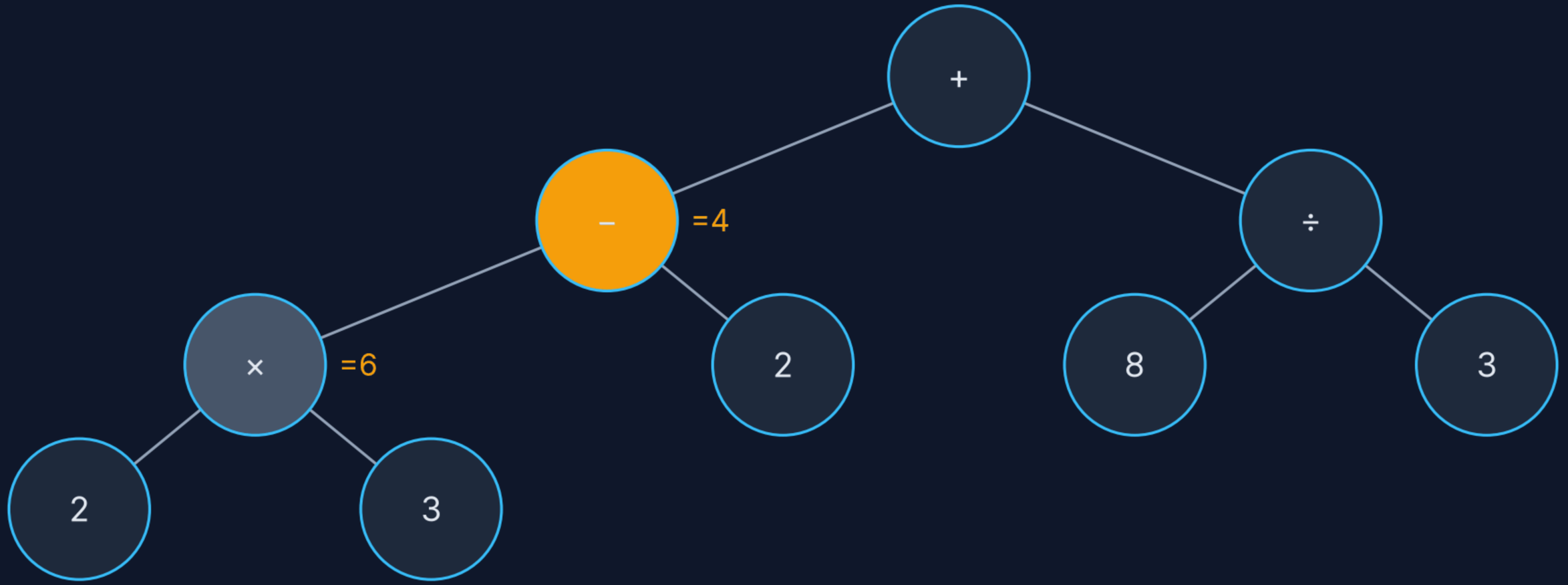
- 1. If the node is a leaf, return its value
- 2. Evaluate left and right subtrees
- 3. Apply the operator encoded at the node
- 4. Division truncates toward zero

Worked example



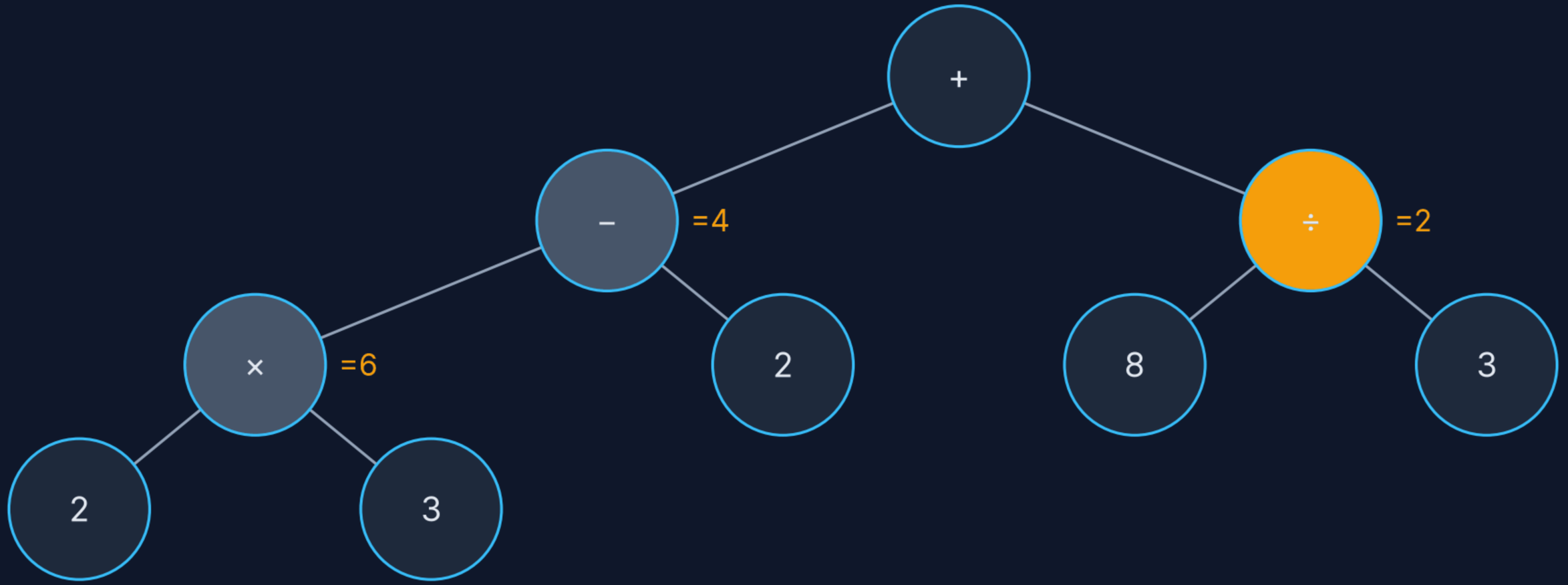
2 × 3 = 6

Worked example



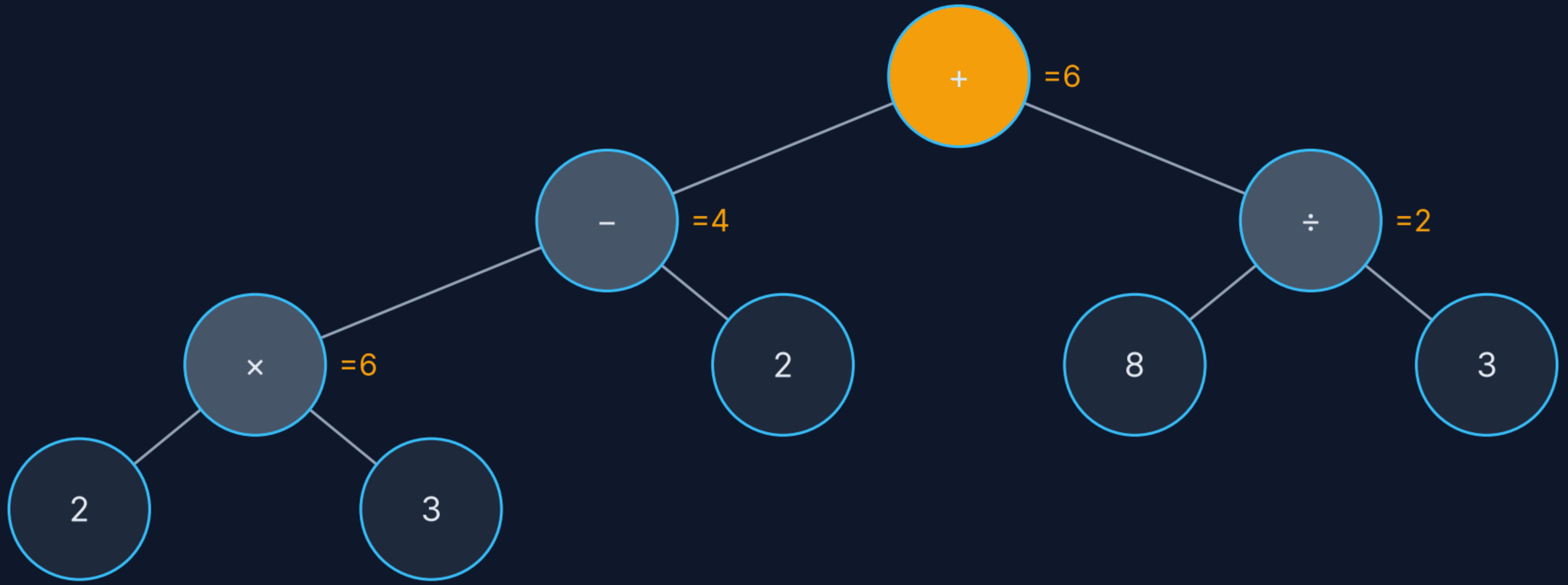
6 − 2 = 4

Worked example



8 ÷ 3 = 2

Worked example



4 + 2 = 6

Java solution

```java
public class EvaluateExpressionTree {
    public static int evaluateExpressionTree(BinaryTree t) {
        if (t.value >= 0) return t.value;
        int l = evaluateExpressionTree(t.left), r = evaluateExpressionTree(t.right);
        return switch (t.value) {
            case -1 -> l + r;
            case -2 -> l - r;
            case -3 -> l / r;
            default -> l * r;
        };
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Division truncates toward zero (−7 / 2 = −3)
- Leaves are non-negative operands
- Avoid recomputing subtrees

Interview tips & follow-ups

- Follow-up: print the infix expression with parentheses
- Follow-up: iterative post-order with a stack

Recap

- Leaves are numbers and internal nodes are operators, so evaluate the children first and combine them.
- Complexity: O(n)